

SELECCIÓN DE FRAMEWORK Y ESTRATEGIA DE VERSIONAMIENTO

MATERIA:

DESARROLLO MÓVIL INTEGRAL

PROFESOR:

DOLORES ABRIL SANCHEZ CAMACHO

**INGENIERÍA EN DESARROLLO DE SOFTWARE
MULTIPLATAFORMA**

PRESENTA:

**BARRIOS NAVA YAIR
HERNANDEZ BARRAGAN CRISTIAN
ESTRELLA BAUTISTA FELIPE
MENDOZA BALDERAS YANELI**

SAN JUAN DEL RÍO, QRO.

ÍNDICE

APARTADO 1 DEFINICIÓN DE NUESTRO PRIMER PROYECTO	5
1. ¿Cuál es el problema?	6
2. ¿Quién lo tiene?	6
3. ¿Cómo se resuelve actualmente?	6
4. ¿Qué problemas tiene la solución actual?	6
5. ¿Cómo podría ayudar una aplicación móvil?.....	6
6. ¿Qué funcionalidades necesitaría una primera versión (MVP)?.....	6
APARTADO 2 PLANEACIÓN EL PROCESO DE DESARROLLO DE ACUERDO A LA METODOLOGÍA SCRUM	8
1. Roles del equipo de trabajo (4 integrantes).....	9
2. Necesidades y requisitos del sistema	9
Requisitos funcionales (RF).....	9
Requisitos no funcionales (RNF).....	10
3. Historias de usuario — Escenarios en Gherkin	11
US-01 (RF-01)	11
US-02 (RF-01)	11
US-03 (RF-02)	11
US-04 (RF-03)	12
US-05 (RF-04)	12
US-06 (RF-05)	12
US-07 (RF-05)	12
US-08 (RF-06)	12
US-09 (RNF-03)	13
US-10 (RNF-04)	13
4. Product Backlog.....	14
5. Organización de los Sprints (10 Sprints, 1 semana c/u)	15
6. Sprint Backlog	16
Sprint 1 — Fundación técnica y diseño base	16
Sprint 2 — Autenticación y control de acceso por rol.....	16
Sprint 3 — Captura móvil de gastos e insumos.....	16
Sprint 4 — Almacenamiento local offline	16
Sprint 5 — Sincronización automática	16

Sprint 6 — Panel móvil de catálogos.....	16
Sprint 7 — Asignación de viajes y anticipos	17
Sprint 8 — Auditoría y validación en ruta.....	17
Sprint 9 — Optimización, vistas adaptables y seguridad	17
Sprint 10 — Liquidaciones, integración final y despliegue.....	17
7. Eventos de Scrum	18
Sprint Planning.....	18
Daily Scrum	18
Sprint Review.....	18
Sprint Retrospective.....	18
8. Incremento por Sprint	19
Sprint 1 — Fundación técnica y diseño base	19
Sprint 2 — Autenticación y control de acceso por rol.....	19
Sprint 3 — Captura móvil de gastos e insumos.....	19
Sprint 4 — Almacenamiento local offline	19
Sprint 5 — Sincronización automática	19
Sprint 6 — Panel móvil de catálogos.....	19
Sprint 7 — Asignación de viajes y anticipos	19
Sprint 8 — Auditoría y validación en ruta.....	19
Sprint 9 — Optimización, vistas adaptables y seguridad	19
Sprint 10 — Liquidaciones, integración final y despliegue.....	19
9. Esquema de pruebas.....	20
Criterio general de aceptación	20
APARTADO 3 ANÁLISIS Y ARQUITECTURA DEL SISTEMA.....	21
1. Preguntas de análisis.....	22
¿La aplicación necesita conexión a Internet? ¿Por qué?	22
¿Necesita usuarios o diferentes tipos de usuarios?	22
¿Necesita almacenar información? ¿Qué información?	22
¿Utilizará una base de datos?	22
¿Necesita consumir o proporcionar una API?	22
¿Utilizará servicios externos?	22
¿Qué componentes principales consideran necesarios para que funcione su aplicación?	22
¿La aplicación necesita funcionar sin conexión a Internet?.....	23
2. Arquitectura seleccionada	23

3. Justificación	23
4. Componentes principales	24
5. Modelado de la arquitectura.....	24
6. ¿Cómo se comunican?	25
APARTADO 4 PATRONES DE DISEÑO	26
1. Preguntas de análisis	27
2. Problema(s) identificado(s).....	28
P1 — Manejo descentralizado del cliente HTTP y la sesión (JWT)	28
P2 — Propagación de cambios de estado a varias pantallas.....	28
P3 — Validación de distintos tipos de gasto con reglas diferentes	28
3. Patrón seleccionado	28
P1 — Singleton	28
P2 — Observer (Publicador-Suscriptor)	28
P3 — Strategy	28
4. Lugar donde se aplicará	29
5. Beneficio esperado.....	29
PARTE 5 FRAMEWORK, HERRAMIENTAS Y ENTORNO DE DESARROLLO.....	30
1. Selección del framework	31
Framework seleccionado	31
Lenguaje de programación utilizado.....	31
Plataformas a las que estará dirigida la aplicación	31
Características principales del framework	31
Ventajas y desventajas	32
Requisitos para trabajar con el framework	32
Justificación de la selección.....	32
2. Herramientas del entorno de desarrollo	33
3. Instalación y configuración	34
1. Requisitos.....	34
2. Instalación	34
3. Configuración	34
4. Verificación.....	35
4. Sistema de control de versiones, plataforma y repositorio.....	36
5. Organización de ramas	36
6. Convención para nombrar ramas	37

7.	Commits.....	37
8.	Flujo de trabajo.....	38
9.	Responsabilidades	38

APARTADO 1 DEFINICIÓN DE NUESTRO PRIMER PROYECTO

1. ¿Cuál es el problema?

Ineficiencia y falta de control en la gestión operativa y financiera del autotransporte de carga. La dependencia de registros manuales en carretera genera opacidad en el consumo de diésel, insumos de mantenimiento y pago a choferes, lo que impide conocer la rentabilidad real de cada viaje y provoca fugas silenciosas de capital.

2. ¿Quién lo tiene?

Empresas de autotransporte de carga federal (SPF) y PyMEs logísticas con flotillas de 5 a 50 unidades. Aunque afecta a flotillas de vans o taxis, el impacto económico es crítico en la carga pesada, donde el diésel representa hasta el 40% del costo operativo y las rutas duran días.

3. ¿Cómo se resuelve actualmente?

Mediante bitácoras de papel, carpetas con notas de consumo y vales físicos que el operador guarda durante el trayecto. Al regresar a la base, el personal de administración recopila los papeles y transcribe los datos manualmente a hojas de cálculo en Excel.

4. ¿Qué problemas tiene la solución actual?

- Pérdida de comprobantes: Extravío de tickets de carga y facturas en ruta.
- Margen de fraude y error humano: Alteración de costos, compras ficticias de insumos o captura incorrecta de números en administración.
- Cálculos tardíos: El dueño conoce las pérdidas o ganancias semanas después de haber concluido el flete.

5. ¿Cómo podría ayudar una aplicación móvil?

Convierte el teléfono del chofer en un escáner de evidencias en tiempo real. Al exigir una foto del ticket, kilometraje y litros cargados en el momento exacto, la app elimina la pérdida de papel, calcula de forma automática el rendimiento (\$km/L\$) y da visibilidad inmediata a la administración sobre el flujo de dinero.

6. ¿Qué funcionalidades necesitaría una primera versión (MVP)?

- Captura de gastos e insumos: Registro express de diésel, aceite y anticongelante con foto obligatoria del ticket y lectura del odómetro.

- Modo Offline: Guardado local de datos cuando no hay cobertura en carretera y sincronización automática al detectar red. Módulo de Viajes y Fletes: Alta de ruta, cliente, tarifa pactada y anticipos entregados.
- Liquidación de Choferes: Cálculo automatizado de pago por kilometraje/comisión menos los anticipos otorgados.
- Panel Web Administrativo: Tablero centralizado para auditar los gastos registrados por los operadores y validar cada viaje.

APARTADO 2 PLANEACIÓN EL PROCESO DE DESARROLLO DE ACUERDO A LA METODOLOGÍA SCRUM

1. Roles del equipo de trabajo (4 integrantes)

Para ejecutar este proyecto bajo el marco de trabajo Scrum con iteraciones semanales, las responsabilidades y tecnologías se distribuyen de forma equilibrada entre los 4 integrantes:

Integrante	Rol	Responsabilidades
Felipe Estrella	Product Owner (PO) & Analista de Negocio	Define la visión del producto, redacta y prioriza las Historias de Usuario en el Product Backlog, establece los Criterios de Aceptación y representa la voz de las empresas de autotransporte. Evalúa y aprueba los incrementos funcionales al final de cada Sprint.
Yair Barrios	Scrum Master (SM) & Desarrollador Lead Mobile	Facilita los eventos de Scrum, elimina bloqueos del equipo y garantiza la disciplina metodológica. Lidera la arquitectura de la aplicación móvil multiplataforma desarrollada en Flutter, enfocada en la experiencia del operador, la captura de fotografías y el almacenamiento local para trabajo sin red.
Cristian Hernández	Desarrollador Backend, Arquitecto de Datos & QA	Diseña y construye la API REST y la base de datos NoSQL en MongoDB para la gestión flexible de fletes, comprobantes y catálogos. Desarrolla el motor de liquidaciones automatizadas, la carga de imágenes a la nube y los endpoints de sincronización offline-online y lidera el plan de pruebas.
Yaneli Mendoza	Diseñadora UX/UI & Frontend	Diseña la arquitectura de información, prototipos y el Sistema de Diseño en Figma para la app móvil. Desarrolla la interfaz del Panel Administrativo para los dueños de flota.

2. Necesidades y requisitos del sistema

Requisitos funcionales (RF)

ID	Nombre	Descripción
RF-01	Autenticación y Control de Acceso por Rol	Inicio de sesión seguro desde la aplicación móvil en Flutter, validando credenciales y redirigiendo según el rol del usuario (Operador para interfaz de ruta o Administrador/Dueño para el panel móvil de control).
RF-02	Captura Móvil de Gastos e Insumos	Registro express para el chofer de cargas de diésel, fluidos (aceite, anticongelante) y viáticos, exigiendo fotografía del comprobante y lectura de odómetro.
RF-03	Almacenamiento Local Offline	Guardado local de datos y fotografías en el dispositivo móvil cuando la unidad transite por tramos carreteros sin red celular.
RF-04	Sincronización Automática	Transmisión en segundo plano de registros guardados offline hacia la base de datos central en MongoDB al detectar señal de internet.
RF-05	Panel Móvil de Catálogos y Fletes	Módulo administrativo en el smartphone para crear/editar camiones, choferes, clientes, tarifas pactadas, asignación de viajes y registro de anticipos.

ID	Nombre	Descripción
RF-06	Auditoría y Validación en Ruta	Vista ejecutiva en la app para que el administrador audite, apruebe o rechace las fotografías de los tickets enviadas por los choferes en tiempo real.

Requisitos no funcionales (RNF)

ID	Nombre	Descripción
RNF-01	Usabilidad y Vistas Adaptables en Flutter	Interfaz desarrollada integralmente en Flutter con dos experiencias diferenciadas: simplificada de alto contraste para el chofer en cabina y una vista de tablero ejecutivo para el administrador desde su teléfono o tablet.
RNF-02	Base de Datos NoSQL y Rendimiento	Implementación de MongoDB para almacenar registros flexibles de rutas y gastos, garantizando respuesta de endpoints en menos de 2 segundos a través de conexiones móviles.
RNF-03	Optimización de Datos Móviles	Compresión automática de imágenes antes de ser almacenadas localmente o enviadas a la nube para preservar el uso de datos del plan celular.
RNF-04	Seguridad y Privacidad	Protocolo de comunicación HTTPS, autenticación con tokens JWT y restricción de permisos para evitar que los operadores accedan al panel financiero de la empresa.

3. Historias de usuario — Escenarios en Gherkin

Cada historia de usuario deriva de un requisito funcional o no funcional y se valida mediante uno o más escenarios en formato Gherkin (Given / When / Then).

US-01 (RF-01)

Como operador (chofer), quiero iniciar sesión con mis credenciales para acceder a la interfaz simplificada de ruta.

Escenario: Inicio de sesión exitoso como Operador

Given que el operador tiene una cuenta registrada en el sistema
And se encuentra en la pantalla de inicio de sesión de la app Flutter
When ingresa su usuario y contraseña válidos
Then el sistema valida sus credenciales
And lo redirige a la interfaz de ruta para operadores

Escenario: Credenciales inválidas

Given que el operador se encuentra en la pantalla de inicio de sesión
When ingresa un usuario o contraseña incorrectos
Then el sistema muestra un mensaje de error
And no permite el acceso a ninguna vista de la aplicación

US-02 (RF-01)

Como administrador/dueño de flota, quiero iniciar sesión y ser dirigido al panel móvil de control para gestionar mi operación.

Escenario: Inicio de sesión exitoso como Administrador

Given que el administrador tiene una cuenta registrada con rol Administrador
When ingresa credenciales válidas en la app
Then el sistema lo redirige al panel administrativo de control

US-03 (RF-02)

Como chofer, quiero registrar de forma express los gastos de diésel, fluidos y viáticos con fotografía del comprobante y lectura del odómetro, para documentar mis consumos en ruta.

Escenario: Registro de carga de diésel con comprobante

Given que el chofer inició sesión en la app
When selecciona "Registrar gasto" y elige el tipo "Diésel"
And captura una fotografía del comprobante
And ingresa la lectura del odómetro
Then el sistema guarda el registro con la fotografía y el odómetro asociados

Escenario: Intento de registro sin fotografía del comprobante

Given que el chofer está registrando un gasto
When intenta guardar el registro sin adjuntar la fotografía del comprobante
Then el sistema rechaza el guardado
And solicita capturar la fotografía antes de continuar

US-04 (RF-03)

Como chofer, quiero que mis registros y fotografías se guarden localmente cuando no tengo señal, para no perder información en tramos carreteros sin cobertura.

Escenario: Registro de gasto sin conexión a red celular

Given que el dispositivo del chofer no tiene señal de red celular
When el chofer registra un gasto con fotografía
Then el sistema guarda el registro y la fotografía localmente en el dispositivo
And marca el registro con el estado "pendiente de sincronización"

US-05 (RF-04)

Como sistema, quiero sincronizar automáticamente en segundo plano los registros guardados offline al recuperar señal, para mantener la base de datos central actualizada sin intervención del chofer.

Escenario: Sincronización al recuperar señal de internet

Given que existen registros con estado "pendiente de sincronización" en el dispositivo
When el dispositivo detecta señal de internet
Then la app transmite en segundo plano los registros pendientes hacia MongoDB
And actualiza el estado de cada registro a "sincronizado"

US-06 (RF-05)

Como administrador, quiero crear y editar camiones, choferes, clientes y tarifas pactadas desde el panel móvil, para mantener actualizados los catálogos de mi operación.

Escenario: Alta de un nuevo camión en el catálogo

Given que el administrador inició sesión en el panel móvil
When accede al módulo "Camiones" y captura los datos del nuevo vehículo
Then el sistema guarda el camión en el catálogo
And lo pone disponible para la asignación de viajes

US-07 (RF-05)

Como administrador, quiero asignar viajes a un camión y chofer disponibles, registrando la tarifa pactada y el anticipo entregado, para controlar los fletes en curso.

Escenario: Asignación de un viaje con anticipo

Given que el administrador tiene un camión y un chofer disponibles
When asigna un viaje indicando cliente, tarifa pactada y monto de anticipo
Then el sistema registra el viaje asignado
And refleja el anticipo entregado al chofer

US-08 (RF-06)

Como administrador, quiero auditar en tiempo real las fotografías de tickets enviadas por los choferes, para aprobar o rechazar los comprobantes antes de la liquidación.

Escenario: Aprobación de un ticket enviado por el chofer

Given que el chofer envió la fotografía de un ticket de gasto
When el administrador revisa la fotografía desde la vista ejecutiva y decide aprobarla

Then el sistema marca el ticket como "aprobado"

And lo refleja en el reporte de liquidación

Escenario: Rechazo de un ticket ilegible

Given que el chofer envió la fotografía de un ticket de gasto

When el administrador revisa la fotografía y detecta que es ilegible

Then el sistema marca el ticket como "rechazado"

And notifica al chofer para volver a capturarlo

US-09 (RNF-03)

Como sistema, quiero comprimir automáticamente las fotografías antes de almacenarlas o enviarlas a la nube, para preservar el uso de datos del plan celular del chofer.

Escenario: Compresión automática de una fotografía capturada

Given que el chofer capturó una fotografía de un comprobante

When la app procesa la imagen antes de guardarla o enviarla

Then el sistema reduce el peso de la imagen conservando la legibilidad

And almacena o transmite la versión comprimida

US-10 (RNF-04)

Como responsable de seguridad del sistema, quiero que la comunicación use HTTPS con tokens JWT y restricción de permisos por rol, para evitar que un operador acceda al panel financiero de la empresa.

Escenario: Un operador intenta acceder al panel financiero

Given que un usuario con rol Operador inició sesión correctamente

When intenta acceder a una ruta del panel financiero del Administrador

Then el sistema rechaza la solicitud por permisos insuficientes

And mantiene el token JWT y la sesión sin exponer datos financieros

4. Product Backlog

Lista priorizada de funcionalidades del sistema, ordenadas de acuerdo con el valor de negocio y la dependencia técnica entre módulos.

ID	Funcionalidad / Historia de usuario	Prioridad	Story Points
US-11	Configuración inicial del proyecto: arquitectura Flutter + Node/Express + MongoDB	Alta	5
US-01	Inicio de sesión y redirección por rol — Operador	Alta	3
US-02	Inicio de sesión y redirección por rol — Administrador	Alta	2
US-03	Captura de gastos con fotografía y odómetro	Alta	5
US-04	Almacenamiento local offline de registros y fotografías	Alta	8
US-05	Sincronización automática offline-online	Alta	8
US-06	Gestión de catálogos (camiones, choferes, clientes, tarifas)	Media	5
US-07	Asignación de viajes y registro de anticipos	Media	5
US-08	Auditoría en ruta — aprobación de tickets	Media	5
US-09comp	Compresión automática de imágenes	Media	3
US-08b	Auditoría en ruta — rechazo de tickets	Baja	2
US-12	Vistas adaptables Flutter (chofer alto contraste / tablero ejecutivo)	Alta	5
US-10	Seguridad: HTTPS, JWT y restricción de permisos por rol	Alta	5
US-13	Motor de liquidaciones automatizadas y reporte de fletes	Media	8

5. Organización de los Sprints (10 Sprints, 1 semana c/u)

El proyecto se planea en 10 Sprints semanales. Cada Sprint entrega un incremento funcional verificable del producto.

Sprint	Semana	Objetivo del Sprint
Sprint 1	Semana 1	Fundación técnica y diseño base
Sprint 2	Semana 2	Autenticación y control de acceso por rol
Sprint 3	Semana 3	Captura móvil de gastos e insumos
Sprint 4	Semana 4	Almacenamiento local offline
Sprint 5	Semana 5	Sincronización automática
Sprint 6	Semana 6	Panel móvil de catálogos
Sprint 7	Semana 7	Asignación de viajes y anticipos
Sprint 8	Semana 8	Auditoría y validación en ruta
Sprint 9	Semana 9	Optimización, vistas adaptables y seguridad
Sprint 10	Semana 10	Liquidaciones, integración final y despliegue

6. Sprint Backlog

Sprint 1 — Fundación técnica y diseño base

- Definir arquitectura del proyecto (Flutter, Node.js/Express, MongoDB)
- Configurar repositorios, ramas y entorno de desarrollo
- Diseñar el modelo de datos inicial en MongoDB (usuarios, camiones, choferes, clientes)
- Elaborar wireframes de baja fidelidad en Figma (login, ruta operador, panel admin)
- Configurar autenticación base con JWT en el backend

Sprint 2 — Autenticación y control de acceso por rol

- Implementar endpoint de login con validación de credenciales (API)
- Implementar pantalla de login en Flutter
- Implementar lógica de redirección según rol (Operador / Administrador)
- Pruebas unitarias del endpoint de autenticación

Sprint 3 — Captura móvil de gastos e insumos

- Diseñar en Figma la pantalla de "Registrar gasto" para el chofer
- Implementar formulario de captura (tipo de gasto, monto, odómetro)
- Integrar captura de fotografía del comprobante con la cámara del dispositivo
- Implementar endpoint para guardar el registro en MongoDB
- Validación de campos obligatorios (foto y odómetro)

Sprint 4 — Almacenamiento local offline

- Integrar base de datos local en el dispositivo (almacenamiento embebido de Flutter)
- Adaptar el flujo de captura de gastos para guardar localmente sin red
- Implementar bandera de estado "pendiente de sincronización"
- Pruebas en dispositivo real en modo avión

Sprint 5 — Sincronización automática

- Implementar servicio en segundo plano para detectar reconexión a internet
- Implementar lógica de envío por lotes de registros pendientes hacia la API
- Manejo de conflictos y reintentos ante fallas de sincronización
- Actualizar estado del registro a "sincronizado" tras confirmación del servidor

Sprint 6 — Panel móvil de catálogos

- Diseñar en Figma el panel administrativo (camiones, choferes, clientes, tarifas)
- Implementar endpoints CRUD de catálogos en la API
- Implementar pantallas de alta/edición de camiones y choferes en Flutter
- Implementar pantallas de alta/edición de clientes y tarifas pactadas

Sprint 7 — Asignación de viajes y anticipos

- Diseñar el flujo de asignación de viaje (camión, chofer, cliente, tarifa)
- Implementar endpoint de asignación de viajes en la API
- Implementar pantalla de asignación de viaje con registro de anticipo
- Vincular el viaje asignado con el catálogo de fletes existente

Sprint 8 — Auditoría y validación en ruta

- Diseñar la vista ejecutiva de auditoría de tickets
- Implementar endpoint para listar tickets pendientes de revisión
- Implementar acciones de aprobar/rechazar ticket con notificación al chofer
- Reflejar el estado del ticket en el reporte de liquidación

Sprint 9 — Optimización, vistas adaptables y seguridad

- Implementar compresión automática de imágenes antes de guardar/enviar (US-09comp)
- Ajustar la interfaz del chofer a un esquema simplificado de alto contraste (RNF-01)
- Ajustar el tablero del administrador para tablet/teléfono (RNF-01)
- Forzar HTTPS en todos los endpoints y reforzar validación de permisos por token JWT (US-10)
- Pruebas de rendimiento: verificar respuesta de endpoints menor a 2 segundos (RNF-02)

Sprint 10 — Liquidaciones, integración final y despliegue

- Implementar el motor de liquidaciones automatizadas (US-13)
- Generar el reporte consolidado de fletes y gastos por chofer/camión
- Ejecutar pruebas de integración end-to-end sobre todos los módulos
- Ejecutar pruebas de aceptación de usuario (UAT) con el Product Owner
- Corrección de bugs detectados y preparación del build de entrega

7. Eventos de Scrum

Dado que cada Sprint dura una semana, los eventos de Scrum se condensan en un ciclo corto y disciplinado, facilitado por Yair Barrios (Scrum Master):

Sprint Planning

Lunes, 45–60 min. El equipo revisa el Product Backlog priorizado por Felipe Estrella (PO) y selecciona las historias de usuario que conformarán el Sprint Backlog de la semana, estimando el esfuerzo en story points.

Daily Scrum

De lunes a viernes, 10–15 min. Cada integrante responde: qué hizo el día anterior, qué hará hoy y si existe algún bloqueo. El Scrum Master da seguimiento a los impedimentos reportados.

Sprint Review

Viernes, 30–45 min. El equipo presenta al Product Owner el incremento funcional obtenido en la semana (app Flutter + API funcionando), y este valida el cumplimiento de los criterios de aceptación de cada historia.

Sprint Retrospective

Viernes, 20–30 min, posterior al Sprint Review. El equipo identifica qué funcionó bien, qué no, y define acciones de mejora para el siguiente Sprint.

8. Incremento por Sprint

Sprint 1 — Fundación técnica y diseño base

Repositorio del proyecto configurado, arquitectura base funcionando end-to-end (app Flutter conectada a una API de prueba en Node.js con MongoDB) y wireframes iniciales aprobados por el Product Owner.

Sprint 2 — Autenticación y control de acceso por rol

US-01 y US-02 completadas: cualquier usuario registrado puede iniciar sesión desde la app y es dirigido automáticamente a la interfaz correspondiente a su rol.

Sprint 3 — Captura móvil de gastos e insumos

US-03 completada: el chofer puede registrar diésel, fluidos y viáticos adjuntando fotografía del comprobante y lectura de odómetro cuando hay conexión.

Sprint 4 — Almacenamiento local offline

US-04 completada: el chofer puede registrar gastos y fotografías sin señal celular; los datos quedan guardados de forma segura en el dispositivo.

Sprint 5 — Sincronización automática

US-05 completada: los registros capturados offline se transmiten automáticamente a MongoDB en cuanto el dispositivo recupera señal, sin intervención del chofer.

Sprint 6 — Panel móvil de catálogos

US-06 completada: el administrador puede crear y editar camiones, choferes, clientes y tarifas desde el panel móvil.

Sprint 7 — Asignación de viajes y anticipos

US-07 completada: el administrador puede asignar viajes a un camión y chofer disponibles, registrando la tarifa pactada y el anticipo entregado.

Sprint 8 — Auditoría y validación en ruta

US-08 y US-08b completadas: el administrador puede revisar en tiempo real las fotografías de tickets enviadas por los choferes, aprobarlas o rechazarlas.

Sprint 9 — Optimización, vistas adaptables y seguridad

US-09comp, US-12 y US-10 completadas: las imágenes se comprimen automáticamente, la interfaz se adapta al rol del usuario y la comunicación queda protegida bajo HTTPS/JWT con permisos restringidos.

Sprint 10 — Liquidaciones, integración final y despliegue

US-13 completada y sistema integrado: la app genera liquidaciones automáticas a partir de los gastos y fletes registrados, con todos los módulos previos integrados, probados y listos para entrega.

9. Esquema de pruebas

El plan de pruebas es liderado por Yaneli Mendoza (QA) con apoyo del equipo completo, y combina distintos tipos de prueba a lo largo de los 10 Sprints:

Tipo de prueba	Alcance	Herramienta / Técnica	Responsable	Cuándo se aplica
Pruebas unitarias	Backend (endpoints de autenticación, gastos, catálogos, fletes, liquidaciones)	Jest + Supertest sobre Node.js/Express	Cristian Hernández	Cada Sprint, sobre el código nuevo
Pruebas de widgets/UI	Pantallas Flutter (login, registro de gastos, panel admin, vista de auditoría)	Flutter Test / Widget Testing	Yaneli Mendoza	Sprints 2, 3, 6, 7, 8, 9
Pruebas de integración	Flujo completo app móvil ↔ API ↔ MongoDB	Postman / Flutter integration_test	Yair Barrios + Cristian Hernández	Sprints 5, 8, 10
Pruebas offline / sincronización	Guardado local sin red y sincronización al recuperar señal	Pruebas manuales en dispositivo físico en modo avión	Yair Barrios	Sprints 4 y 5
Pruebas de rendimiento	Tiempo de respuesta de endpoints (< 2 segundos)	Apache JMeter / Postman	Cristian Hernández	Sprint 9
Pruebas de seguridad	HTTPS, tokens JWT, restricción de acceso por rol	Pruebas manuales de penetración básica + revisión de código	Yair Barrios	Sprint 9
Pruebas de usabilidad	Interfaz de alto contraste para el chofer y tablero ejecutivo para el administrador	Sesiones guiadas con usuarios reales del sector autotransporte	Yaneli Mendoza	Sprints 3, 6, 9
Pruebas de aceptación (UAT)	Validación de todo el sistema contra los criterios de aceptación del Product Owner	Sesión de demostración guiada con checklist de criterios de aceptación	Felipe Estrella (PO) + equipo completo	Sprint 10

Criterio general de aceptación

Una historia de usuario se considera terminada ("Done") cuando: (1) el código pasa las pruebas unitarias correspondientes, (2) la funcionalidad fue probada manualmente en un dispositivo Android/iOS real, (3) cumple todos los escenarios Gherkin definidos, y (4) el Product Owner la valida en el Sprint Review.

APARTADO 3 ANÁLISIS Y ARQUITECTURA DEL SISTEMA

1. Preguntas de análisis

¿La aplicación necesita conexión a Internet? ¿Por qué?

Sí, pero de forma parcial. El módulo del Administrador (catálogos, asignación de viajes, auditoría en tiempo real, liquidaciones) requiere conexión para consultar y actualizar la información central. El módulo del Operador (chofer) necesita Internet únicamente para sincronizar los registros, no para capturarlos, ya que gran parte de las rutas de autotransporte cruzan tramos carreteros sin cobertura celular.

¿Necesita usuarios o diferentes tipos de usuarios?

Sí. El sistema define dos roles (RF-01): Operador (chofer), con acceso a la interfaz simplificada de captura de gastos en ruta, y Administrador/Dueño de flota, con acceso al panel de catálogos, fletes, auditoría y liquidaciones. El acceso a cada vista y a cada endpoint se restringe según el rol autenticado.

¿Necesita almacenar información? ¿Qué información?

Sí. Se almacena: usuarios y roles; catálogos de camiones, choferes, clientes y tarifas pactadas; registros de gastos e insumos (tipo, monto, fotografía del comprobante, lectura de odómetro); viajes/fletes asignados y anticipos entregados; tickets enviados para auditoría con su estado (pendiente/aprobado/rechazado); y las liquidaciones generadas a partir de los gastos y fletes.

¿Utilizará una base de datos?

Sí. Se usa MongoDB (NoSQL) como base de datos central en el servidor, por la naturaleza flexible y semi-estructurada de los registros (fletes, comprobantes, catálogos). Adicionalmente, el dispositivo móvil usa una base de datos local embebida para operar sin conexión.

¿Necesita consumir o proporcionar una API?

Ambas. La aplicación Flutter consume una API REST propia, desarrollada en Node.js/Express, que expone los recursos de autenticación, gastos, catálogos, fletes, auditoría y liquidaciones. Esa misma API es la que el equipo backend proporciona y mantiene.

¿Utilizará servicios externos?

Sí. Se contempla un servicio externo de almacenamiento en la nube (por ejemplo Firebase Storage o Amazon S3) para alojar las fotografías de comprobantes y tickets, evitando saturar la base de datos con archivos binarios.

¿Qué componentes principales consideran necesarios para que funcione su aplicación?

App móvil Flutter (cliente), base de datos local embebida para modo offline, un servicio de sincronización en segundo plano, una API REST (Node.js/Express) con middleware de seguridad (JWT/HTTPS), una base de datos central en MongoDB y un servicio de almacenamiento en la nube para imágenes.

¿La aplicación necesita funcionar sin conexión a Internet?

Sí, de forma parcial y crítica (RF-03). El flujo de captura de gastos del chofer debe operar completamente offline: el registro y la fotografía se guardan localmente y quedan marcados como "pendientes de sincronización" hasta que el dispositivo recupera señal (RF-04), momento en el que se transmiten automáticamente al servidor.

2. Arquitectura seleccionada

Arquitectura cliente-servidor en 3 capas (presentación, lógica de negocio y datos), con el cliente móvil implementado bajo un enfoque Offline-First y sincronización diferida (deferred sync).

- Capa de presentación: app móvil multiplataforma en Flutter, con dos interfaces diferenciadas por rol (Operador / Administrador).
- Capa de lógica de negocio: API REST en Node.js/Express, con autenticación JWT y comunicación cifrada por HTTPS.
- Capa de datos: base de datos central MongoDB (NoSQL), complementada con almacenamiento local embebido en el dispositivo y un servicio externo de almacenamiento de imágenes en la nube.

3. Justificación

Como señala la consigna, una arquitectura no se elige porque "es la mejor" en abstracto, sino porque responde a las necesidades reales del proyecto. Para el SPF, la elección se justifica así:

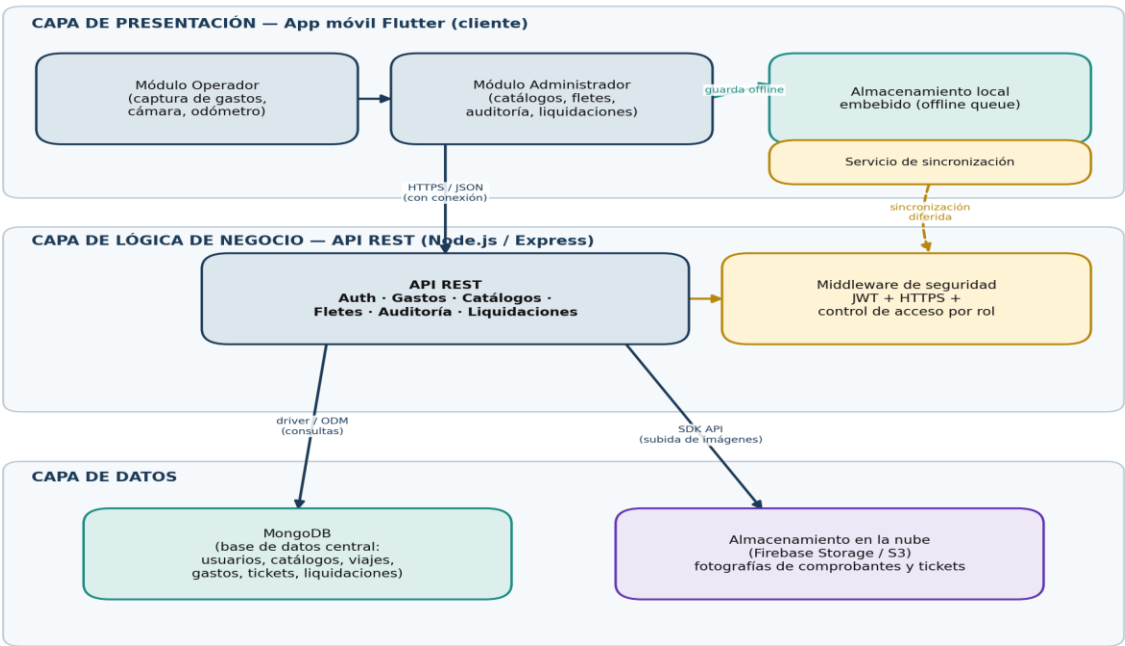
- Offline-First en el cliente: las unidades de autotransporte recorren tramos carreteros sin cobertura celular, por lo que el chofer necesita registrar gastos sin depender de la red (RF-03); una arquitectura puramente "online-only" haría inviable el caso de uso principal del operador.
- Cliente-servidor de 3 capas: separa con claridad la interfaz (Flutter), las reglas de negocio (API) y el almacenamiento (MongoDB + nube), lo que permite que cada integrante del equipo trabaje su capa de forma independiente (Mobile: Yair, Backend: Cristian, UX/QA: Yaneli) sin bloquearse entre sí, algo clave en Sprints semanales.
- API REST con Node.js/Express: es un estándar ligero, ampliamente soportado, que se integra de forma natural con Flutter mediante peticiones HTTP/JSON y permite versionar y reutilizar los mismos endpoints si en el futuro se agrega un panel web administrativo.
- MongoDB (NoSQL): los documentos de fletes, gastos y tickets varían en estructura según el tipo de gasto o el estado del ticket; un esquema flexible tipo documento se adapta mejor a esa variabilidad que una base de datos relacional rígida, además de facilitar la sincronización de lotes de registros offline.
- JWT + HTTPS: al ser una app móvil sin sesiones de servidor, un token JWT permite autenticar cada petición de forma stateless y aplicar el control de acceso por rol exigido en RNF-04 (evitar que el operador acceda al panel financiero).
- Almacenamiento externo de imágenes: separar las fotografías (archivos binarios pesados) de la base de datos evita degradar el rendimiento de las consultas en MongoDB y facilita cumplir el RNF-02 (respuesta de endpoints menor a 2 segundos).

4. Componentes principales

Componente	Descripción
App móvil Flutter — Módulo Operador	Interfaz simplificada de alto contraste para captura de gastos, fotografía de comprobantes y lectura de odómetro.
App móvil Flutter — Módulo Administrador	Tablero ejecutivo para catálogos, asignación de fletes, auditoría de tickets y liquidaciones.
Base de datos local embebida	Almacena registros y fotografías en el dispositivo cuando no hay conexión; funciona como cola de sincronización.
Servicio de sincronización	Detecta la reconexión a Internet y transmite en segundo plano los registros pendientes hacia la API.
API REST (Node.js/Express)	Expone los recursos de autenticación, gastos, catálogos, fletes, auditoría y liquidaciones; aplica las reglas de negocio.
Middleware de seguridad	Validación de tokens JWT, forzado de HTTPS y control de acceso por rol (Operador/Administrador).
Base de datos central (MongoDB)	Almacena de forma persistente usuarios, catálogos, viajes, gastos, tickets y liquidaciones.
Almacenamiento en la nube de imágenes	Servicio externo (Firebase Storage / S3) donde se alojan las fotografías; MongoDB guarda solo la URL/referencia.

5. Modelado de la arquitectura

Diagrama de componentes que representa visualmente los módulos del sistema y su distribución en capas:



6. ¿Cómo se comunican?

- App Flutter ↔ API REST: comunicación por HTTPS con mensajes en formato JSON, autenticada con un token JWT (Bearer) obtenido en el login; cada endpoint valida el rol del usuario antes de responder.
- App Flutter ↔ Base de datos local: cuando no hay conexión, la app lee y escribe directamente en el almacenamiento embebido del dispositivo, sin pasar por la red.
- Base de datos local ↔ API REST (sincronización diferida): el servicio de sincronización detecta la señal de Internet y reenvía, mediante las mismas peticiones HTTPS/JSON, los registros marcados como "pendientes de sincronización"; al confirmar el servidor, el registro local se marca como "sincronizado".
- API REST ↔ MongoDB: comunicación mediante el driver/ODM de MongoDB (protocolo nativo), usada para leer y escribir catálogos, fletes, gastos, tickets y liquidaciones.
- API REST ↔ Almacenamiento en la nube: la API sube las fotografías (comprobantes y tickets) al servicio externo mediante su SDK/API, y guarda únicamente la URL resultante en el documento correspondiente de MongoDB.

APARTADO 4 PATRONES DE DISEÑO

1. Preguntas de análisis

Para cada problema detectado en el sistema se respondieron las cuatro preguntas guía antes de elegir el patrón:

ID	Parte difícil de mantener/extender	Patrón	Por qué ese patrón y no otro	Componente
P1	El cliente HTTP y el manejo del token JWT / estado de sesión se consumen desde varias pantallas (login, captura de gastos, panel administrativo, servicio de sincronización).	Singleton	El problema es garantizar una única instancia compartida y un punto de acceso global controlado, no un comportamiento intercambiable (Strategy) ni una jerarquía de creación de objetos (Factory).	Servicio "ApiClient / SessionManager" en la capa de comunicación de la app Flutter.
P2	La propagación de cambios de estado de un registro (gasto "pendiente de sincronización" → "sincronizado", o ticket "pendiente" → "aprobado"/"rechazado") hacia varias partes de la app que necesitan enterarse.	Observer (Publicador-Suscriptor)	El problema es de tipo "uno a muchos": un evento debe propagarse a varios interesados sin que quien lo emite conozca a cada uno de ellos. Ni Singleton (una sola instancia) ni Strategy (comportamiento intercambiable) resuelven la propagación de eventos.	Servicio de sincronización y módulo de auditoría de tickets de la app Flutter.
P3	La validación y el registro de gastos (RF-02) para distintos tipos: diésel, fluidos (aceite, anticongelante) y viáticos, cada uno con reglas de validación distintas.	Strategy	El problema es un mismo proceso (validar/registrar un gasto) que cambia de algoritmo según el tipo de gasto; Strategy encapsula cada regla de validación en una clase intercambiable, a diferencia de Singleton (una instancia) u Observer (propagación de eventos), que no resuelven un comportamiento variable.	Módulo de captura de gastos en la app Flutter (formulario dinámico según tipo de gasto) y el controlador del endpoint /gastos en la API REST.

2. Problema(s) identificado(s)

Se identificaron tres problemas reales del sistema SPF susceptibles de resolverse con un patrón de diseño:

P1 — Manejo descentralizado del cliente HTTP y la sesión (JWT)

Si cada pantalla crea su propia instancia del cliente HTTP o administra su propio token JWT, se corre el riesgo de manejar tokens desincronizados entre módulos, disparar varios intentos de reconexión simultáneos y duplicar la lógica de cabeceras HTTPS/JWT. Esto es difícil de mantener y aumenta el riesgo de errores de seguridad (RNF-04) conforme la app crece y se agregan más pantallas.

P2 — Propagación de cambios de estado a varias pantallas

Cuando cambia el estado de un gasto o de un ticket, deben actualizarse el badge de pendientes en la pantalla del chofer, la lista de auditoría del administrador y el propio servicio de sincronización. Si el servicio de sincronización llama directamente a cada una de estas partes, el código queda fuertemente acoplado y cada vez que se agregue una nueva pantalla interesada en ese cambio habrá que modificar el servicio de sincronización, lo que dificulta su extensión y mantenimiento.

P3 — Validación de distintos tipos de gasto con reglas diferentes

Diésel exige lectura de odómetro, mientras que viáticos no; si esta lógica se implementa con un bloque de condicionales (if/switch) dentro de la pantalla de captura o del controlador de la API, cada nuevo tipo de gasto que se agregue obliga a modificar ese mismo bloque, aumentando el riesgo de romper la validación de los tipos ya existentes.

3. Patrón seleccionado

P1 — Singleton

El patrón Singleton garantiza que exista una única instancia del cliente HTTP/administrador de sesión en toda la app, con un punto de acceso global controlado. Se eligió sobre otros patrones creacionales porque aquí no se necesita variar el tipo de objeto creado (como en Factory Method), sino asegurar que solo exista una instancia compartida y consistente.

P2 — Observer (Publicador-Suscriptor)

El patrón Observer permite que el servicio de sincronización (el "sujeto") emita el evento de cambio de estado sin conocer a sus interesados; las pantallas (badge, lista de pendientes, vista de auditoría) se suscriben como "observadores" y reaccionan de forma independiente. Se eligió sobre otras opciones porque resuelve directamente la propagación desacoplada de un evento hacia múltiples receptores.

P3 — Strategy

El patrón Strategy encapsula cada regla de validación de un tipo de gasto en una clase independiente e intercambiable, seleccionada en tiempo de ejecución según el tipo elegido por el chofer. Se eligió porque el problema es de comportamiento variable ante un mismo proceso, exactamente lo que Strategy resuelve, respetando el principio abierto/cerrado.

4. Lugar donde se aplicará

Patrón	Lugar de aplicación
Singleton	Servicio "ApiClient / SessionManager" (capa de comunicación de la app Flutter), usado por los módulos Operador, Administrador y el servicio de sincronización.
Observer Suscriptor) (Publicador-	Módulo de sincronización (emite el evento al cambiar el estado de un registro) y módulo de auditoría de tickets (emite el evento al aprobar/rechazar); las pantallas de badge, lista de pendientes y auditoría actúan como observadores.
Strategy	Módulo de captura de gastos, tanto en la app Flutter (formulario dinámico) como en el controlador del endpoint /gastos de la API REST (Node.js/Express).

5. Beneficio esperado

Patrón	Beneficio esperado
Singleton	Un único punto de verdad para el token JWT y la configuración de conexión; se evita duplicar llamadas de refresco de sesión y se centraliza el manejo de errores de autenticación (por ejemplo, un 401 que expira la sesión en toda la app a la vez).
Observer Suscriptor) (Publicador-	Bajo acoplamiento entre el servicio de sincronización/auditoría y las pantallas que dependen de ese estado; agregar una nueva vista interesada en los cambios no requiere modificar el servicio emisor, solo suscribir un nuevo observador.
Strategy	Agregar un nuevo tipo de gasto en el futuro (por ejemplo "peajes") solo implica crear una nueva estrategia de validación, sin modificar el código existente ni arriesgar la validación de los tipos ya implementados.

PARTE 5 FRAMEWORK, HERRAMIENTAS Y ENTORNO DE DESARROLLO

1. Selección del framework

Framework seleccionado

Flutter, el SDK de código abierto de Google para desarrollo de interfaces multiplataforma.

Lenguaje de programación utilizado

Dart, lenguaje orientado a objetos desarrollado también por Google, compilado tanto a código nativo (ARM/x64) como a JavaScript para web.

Plataformas a las que estará dirigida la aplicación

Android e iOS como plataformas principales (choferes y administradores usando smartphone o tablet). Flutter permite, con el mismo código, compilar a Web y Escritorio si el proyecto lo requiriera más adelante.

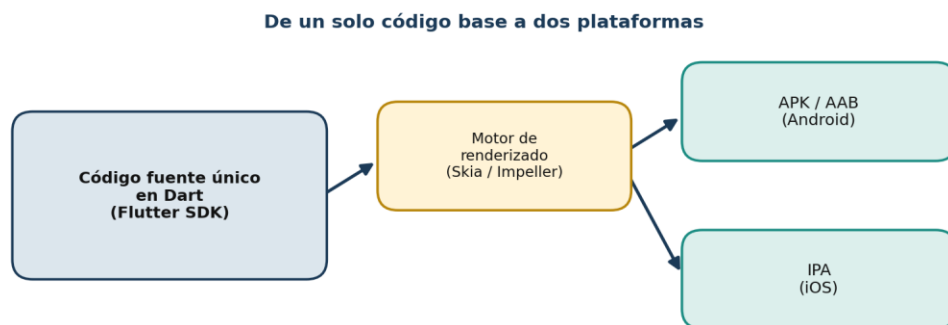


Figura 1. Un mismo código Dart compila a Android (APK/AAB) e iOS (IPA).

Características principales del framework

- Hot reload: los cambios de código se reflejan en la app en segundos, sin reiniciar la aplicación completa.
- Widgets propios: Flutter dibuja su propia interfaz (no reutiliza componentes nativos del sistema operativo), logrando apariencia y comportamiento consistentes en Android e iOS.
- Un solo código base para múltiples plataformas, reduciendo el tiempo de desarrollo frente a mantener dos apps nativas separadas.
- Motor de renderizado propio (Skia / Impeller), que dibuja directamente sobre el canvas del dispositivo.
- Amplio ecosistema de paquetes en pub.dev para cámara, almacenamiento local, peticiones HTTP, manejo de estado, etc.

Ventajas y desventajas

Ventajas	Desventajas
Un solo código para Android e iOS: menor tiempo de desarrollo con 4 integrantes y Sprints semanales.	El tamaño del binario final suele ser mayor que el de una app nativa equivalente.
Hot reload acelera notablemente el ciclo de prueba durante cada Sprint.	Algunas funciones muy específicas del sistema operativo requieren código nativo adicional (platform channels).
Gran cantidad de paquetes ya probados para cámara, almacenamiento local (sqflite/Hive) y HTTP.	Curva de aprendizaje del lenguaje Dart para quien no lo haya usado antes.
Comunidad amplia y respaldo directo de Google, con documentación oficial extensa.	El uso de widgets propios en vez de componentes nativos puede añadir ligera sobrecarga de renderizado en dispositivos de gama muy baja.

Requisitos para trabajar con el framework

- Flutter SDK (incluye el compilador Dart) instalado y agregado al PATH del sistema.
- Android Studio con Android SDK, para compilar y ejecutar en emulador/dispositivo Android.
- Un editor de código con soporte para Flutter/Dart (VS Code o Android Studio).
- Un dispositivo físico en modo desarrollador o un emulador configurado.
- Git, para el control de versiones del proyecto.

Justificación de la selección

Flutter se seleccionó porque encaja directamente con las necesidades del SPF: se necesita una sola app que funcione en los teléfonos Android de los choferes y, potencialmente, en iOS para los administradores, sin duplicar el desarrollo en dos lenguajes nativos distintos dado que el equipo es de solo 4 integrantes con un Sprint semanal. El hot reload acelera la iteración de la interfaz de captura de gastos y del panel administrativo en cada Sprint, y el ecosistema de paquetes de Flutter ya resuelve necesidades concretas del proyecto: acceso a la cámara (RF-02), almacenamiento local embebido para el modo offline (RF-03) y peticiones HTTPS hacia la API REST (RF-04, RF-05, RF-06). En conjunto, Flutter no se eligió por ser "el más popular", sino porque su modelo de un solo código base resuelve directamente la restricción de tiempo y de equipo reducido del proyecto.

2. Herramientas del entorno de desarrollo

Herramientas necesarias para desarrollar, probar y ejecutar la aplicación móvil y su backend:

Herramienta	Versión	¿Para qué sirve?	¿Por qué es necesaria?
Flutter SDK	3.38.5 (canal stable)	Compilar, ejecutar y depurar la app móvil; incluye el SDK de Dart.	Es el framework elegido para construir toda la interfaz de la app (RF-01 a RF-06).
Android Studio	2026.1.4	Proveer el Android SDK, el emulador y las herramientas de compilación para Android.	Necesario para compilar/probar la app en dispositivos y emuladores Android.
Visual Studio Code	1.107.1 extensiones Flutter/Dart +	Editor principal de código para la app Flutter y el backend en Node.js.	Ofrece autocompletado, depurador integrado y hot reload directo desde el editor.
Node.js	16.20.2	Ejecutar el servidor de la API REST desarrollada en JavaScript.	Es la base de ejecución del backend (Express) que expone los endpoints del sistema.
npm	8.19.45	Instalar y administrar las dependencias del backend (Express, Mongoose, jsonwebtoken, etc.).	Se usa en cada Sprint para agregar librerías nuevas del backend.
MongoDB Community Server	8.2.3	Motor de base de datos NoSQL que almacena catálogos, gastos, fletes, tickets y liquidaciones.	Es la base de datos central seleccionada en la arquitectura (Apartado 3).
MongoDB Compass	1.49.4	Interfaz gráfica para explorar, consultar y depurar las colecciones de MongoDB.	Permite verificar visualmente los datos sincronizados sin escribir consultas manuales.
Postman	11.x	Probar manualmente los endpoints de la API REST antes de integrarlos en la app.	Se usa en las pruebas de integración descritas en el esquema de pruebas (Apartado 2).
Git y GitHub	2.52.0.windows.1	Control de versiones y repositorio remoto compartido del proyecto.	Permite que los 4 integrantes trabajen en ramas independientes por módulo cada Sprint.
JDK (Java Development Kit)	25.0.2LTS	Herramientas de compilación requeridas internamente por el Android SDK.	Sin el JDK, Android Studio no puede compilar el proyecto Android generado por Flutter.

3. Instalación y configuración

1. Requisitos

- Sistema operativo de 64 bits (Windows 10/11, macOS o Linux) con al menos 8 GB de RAM y 10 GB de espacio libre.
- Conexión a Internet estable para descargar los SDK, dependencias de npm y paquetes de pub.dev.
- Cuenta de GitHub con acceso al repositorio del equipo.
- Permisos de administrador en el equipo para instalar el Android SDK y las variables de entorno.

2. Instalación

- Descargar e instalar el Flutter SDK desde el canal stable y descomprimirlo en una ruta sin espacios ni acentos.
- Instalar Android Studio y, desde su SDK Manager, agregar el Android SDK, el Android SDK Platform-Tools y una imagen de sistema para el emulador.
- Instalar Visual Studio Code y agregar las extensiones oficiales "Flutter" y "Dart".
- Instalar Node.js LTS (incluye npm) y, dentro de la carpeta del backend, ejecutar npm install para descargar Express, Mongoose y las demás dependencias.
- Instalar MongoDB Community Server y, de forma opcional, MongoDB Compass como interfaz gráfica.
- Instalar Postman y Git.

3. Configuración

- Agregar la carpeta bin del Flutter SDK a la variable de entorno PATH del sistema operativo.
- Ejecutar flutter config --android-sdk apuntando a la ruta del Android SDK instalado por Android Studio.
- Configurar en VS Code las extensiones de Flutter y Dart, y seleccionar el dispositivo/emulador de destino en la barra inferior del editor.
- Crear el archivo .env del backend con la variable MONGODB_URI apuntando a la instancia local o a un clúster de MongoDB Atlas, y una variable JWT_SECRET para firmar los tokens de sesión.
- Configurar en MongoDB Compass una nueva conexión hacia la base de datos spf_autotransporte.

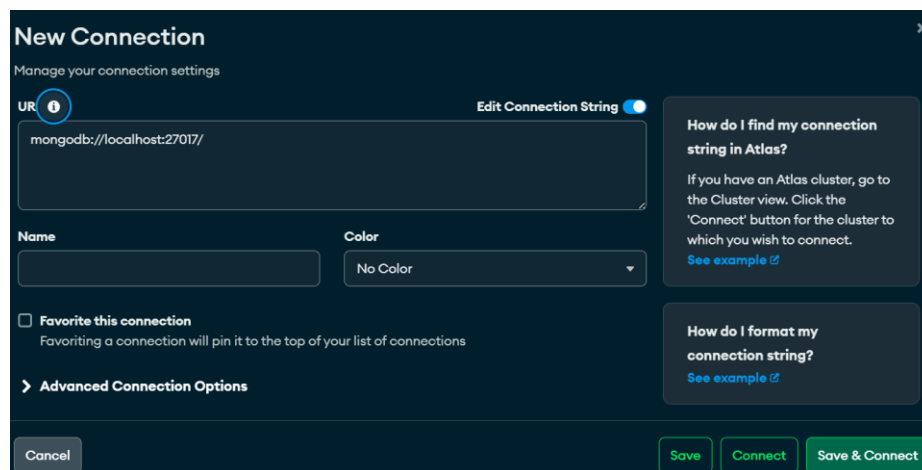


Figura 3. Conexión configurada en MongoDB Compass hacia la base de datos del proyecto.

4. Verificación

Una vez instalado y configurado el entorno, se verificó que cada herramienta respondiera correctamente desde la terminal:

```
Microsoft Windows [Versión 10.0.26200.9168]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\felip>flutter doctor -v

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

[✓] Flutter (Channel stable, 3.38.5, on Microsoft Windows [Versión 10.0.26200.9168], locale es-MX) [339ms]
    • Flutter version 3.38.5 on channel stable at C:\src\flutter
    • Upstream repository https://github.com/flutter/flutter.git
    • Framework revision f6ff1529fd (9 months ago), 2025-12-11 11:50:07 -0500
    • Engine revision 1527ae8ec5
    • Dart version 3.10.4
    • DevTools version 2.51.1
    • Feature flags: enable-web, enable-linux-desktop, enable-macos-desktop, enable-windows-desktop, enable-android,
      enable-ios, cli-animations, enable-native-assets, omit-legacy-version-file, enable-lldb-debugging

[✓] Windows Version (Windows 11 or higher, 25H2, 2009) [784ms]

[✓] Android toolchain - develop for Android devices (Android SDK version 36.1.0) [3.1s]
    • Android SDK at C:\Users\felip\AppData\Local\Android\sdk
    • Emulator version 36.3.10.0 (build_id 14472402) (CL:N/A)
    • Platform android-36, build-tools 36.1.0
    • Java binary at: C:\Program Files\Android\Android Studio\jbr\bin\java
    • This is the JDK bundled with the latest Android Studio installation on this machine.
    • To manually set the JDK path, use: "flutter config --jdk-dir="path/to/jdk"
    • Java version OpenJDK Runtime Environment (build 21.0.8+14196175-b1038.72)
    • All Android licenses accepted.

[✓] Chrome - develop for the web [150ms]
    • Chrome at C:\Program Files\Google\Chrome\Application\chrome.exe

[✓] Visual Studio - develop Windows apps (Visual Studio Community 2026 18.8.1) [149ms]
    • Visual Studio at C:\Program Files\Microsoft Visual Studio\18\Community
    • Visual Studio Community 2026 version 18.8.12021.73
    • Windows 10 SDK version 10.0.26100.0

[✓] Connected device (3 available) [2.3s]
    • Windows (desktop) • windows • windows-x64 • Microsoft Windows [Versión 10.0.26200.9168]
    • Chrome (web) • chrome • web-javascript • Google Chrome 151.0.7922.175
    • Edge (web) • edge • web-javascript • Microsoft Edge 153.0.4234.32

[✓] Network resources [312ms]
    • All expected network resources are available.

• No issues found!

C:\Users\felip>
```

Figura 4. Salida de "flutter doctor -v" confirmando que el entorno Flutter está listo (SDK, Android toolchain, editor y dispositivo conectado).

```
C:\Users\felip>node -v
v16.20.2

C:\Users\felip>npm -v
8.19.4

C:\Users\felip>mongod --version
db version v8.2.3
Build Info: {
  "version": "8.2.3",
  "gitVersion": "36f41c9c30a2f13f834d033ba03c3463c891fb01",
  "modules": [],
  "allocator": "tcmalloc-gperf",
  "environment": {
    "distmod": "windows"
  }
}

C:\Users\felip>git --version
git version 2.52.0.windows.1

C:\Users\felip>
```

Figura 5. Verificación de versiones de Node.js, npm, MongoDB y Git en la terminal del backend.

Con estas verificaciones exitosas, el entorno quedó listo para comenzar el Sprint 1 (configuración inicial del proyecto) descrito en la planeación Scrum.

4. Sistema de control de versiones, plataforma y repositorio

Aspecto	Definición
Sistema de control de versiones	Git.
Plataforma de versionamiento	GitHub.
Repositorio	control-fletes-devops, repositorio privado del equipo bajo la cuenta de GitHub del proyecto.
Enlace de acceso	[Pendiente: pegar aquí el enlace real del repositorio, por ejemplo https://github.com/Cristian2040/control-fletes-devops , en cuanto el equipo lo cree]

5. Organización de ramas

El equipo utiliza un flujo tipo Gitflow simplificado, adecuado para Sprints semanales con 4 integrantes:

Rama	Uso
main	Contiene siempre la versión estable y funcional del proyecto, lista para presentar en el Sprint Review. Solo recibe cambios a través de una rama release o un hotfix, nunca directamente.
develop	Rama de integración donde se combinan las funcionalidades terminadas de cada Sprint antes de pasar a main. Es la base sobre la que se crean todas las ramas feature.
feature/*	Una rama por cada historia de usuario o tarea del Sprint Backlog (por ejemplo feature/rf02-captura-gastos). Se crea desde develop y se fusiona de vuelta a develop al terminar.
release/*	Se crea desde develop al cierre de un Sprint que entrega un incremento listo para revisión (por ejemplo release/sprint-10); permite ajustes finales antes de fusionarse a main y a develop.
fix/*	Se usa para corregir errores urgentes detectados después de haber fusionado a main; se crea desde main y se fusiona de vuelta a main y a develop.

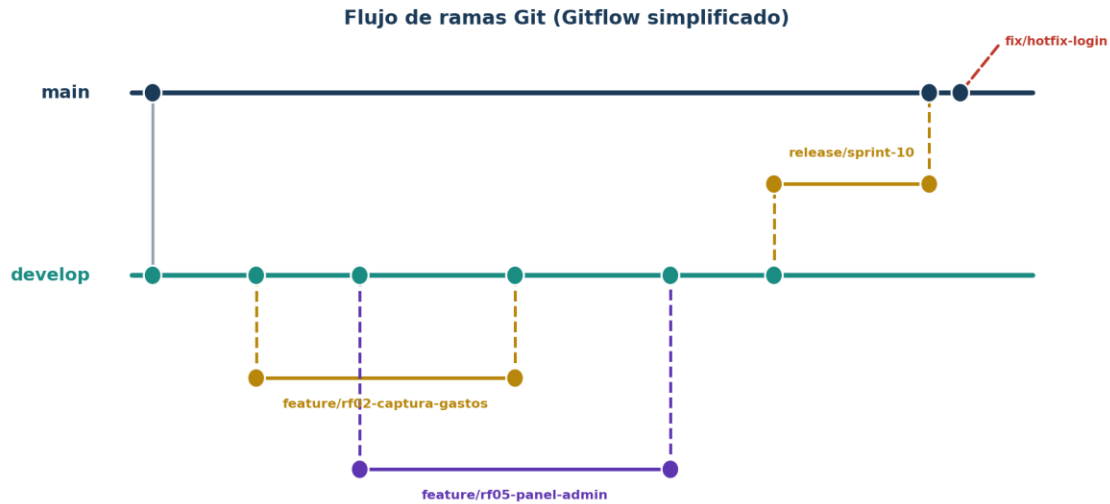


Figura 6. Flujo de ramas del proyecto: main, develop, feature/*, release/* y fix/*.

6. Convención para nombrar ramas

Regla: tipo/descripcion-corta-en-kebab-case, opcionalmente con el identificador del requisito o historia de usuario relacionada.

- feature/<rf-o-historia>-<descripcion-corta> → feature/rf03-almacenamiento-offline
- release/sprint-<numero> → release/sprint-7
- fix/<descripcion-corta> → fix/sync-duplicado-gastos

Todo en minúsculas, sin espacios ni acentos, usando guiones para separar palabras.

7. Commits

Se sigue la convención Conventional Commits: tipo(alcance): descripción breve en presente, para que el historial sea legible y fácil de rastrear por Sprint.

Tipo	Uso	Ejemplo
feat	Nueva funcionalidad	feat(gastos): agrega captura de fotografía del comprobante
fix	Corrección de errores	fix(sync): corrige duplicidad al sincronizar registros offline
docs	Cambios de documentación	docs(readme): agrega instrucciones de instalación
refactor	Cambios internos sin alterar el comportamiento	refactor(auth): simplifica el middleware de JWT
test	Agregar o corregir pruebas	test(catalogos): agrega pruebas unitarias de alta de camión

Tipo	Uso	Ejemplo
chore	Tareas de mantenimiento (dependencias, configuración)	chore(deps): actualiza mongoose a la versión 8

8. Flujo de trabajo

Pasos que sigue cualquier integrante del equipo al realizar un cambio:

- 1. Tomar una tarea del Sprint Backlog de la semana en curso.
- 2. Actualizar su copia local de develop (git pull) para partir del código más reciente.
- 3. Crear una rama feature/* a partir de develop, siguiendo la convención de nombres.
- 4. Desarrollar el cambio realizando commits pequeños y frecuentes, siguiendo la convención de commits.
- 5. Subir la rama al repositorio remoto (git push) y abrir un Pull Request hacia develop.
- 6. Esperar la revisión de código de al menos un compañero antes de fusionar.
- 7. Atender los comentarios de la revisión y actualizar el Pull Request si es necesario.
- 8. Una vez aprobado, fusionar el Pull Request a develop y eliminar la rama feature/* ya integrada.
- 9. Al cierre del Sprint, crear la rama release/sprint-N desde develop y, tras validarla en el Sprint Review, fusionarla a main y de vuelta a develop.

9. Responsabilidades

Responsabilidad	Responsable(s)
Crear Pull Requests	Cada integrante, al terminar la tarea que le corresponde del Sprint Backlog (Felipe, Yair, Cristian y Yaneli).
Revisar Pull Requests / revisar código	Revisión cruzada por módulo: Cristian Hernández revisa los cambios de backend/API, Yair Barrios revisa los cambios de la app Flutter, y Yaneli Mendoza revisa la interfaz, la experiencia de usuario y la cobertura de pruebas. Felipe Estrella (PO) revisa que el resultado cumpla los criterios de aceptación de la historia de usuario.
Integrar cambios (merge)	Yair Barrios (Scrum Master), una vez que el Pull Request cuenta con al menos una aprobación.
Mantener la rama principal (main)	Yair Barrios como responsable directo, con apoyo de Cristian Hernández para validar que el backend y la base de datos queden estables antes de cada fusión a main.
Resolver conflictos	En primera instancia, el integrante que generó el conflicto junto con el revisor asignado a esa rama; si el conflicto involucra a varios módulos, Yair Barrios coordina la resolución en la Daily Scrum más cercana.